Ders 20 Çalışma Özeti

Ders 20 Çalışma Özeti

Genel Konular

- Insertion sort (ekrandan, 00:30:00)
  - Mantık: kart dizmeye benzer. İkinci elemandan başlanır. Her eleman, solundaki sıralı kısımda olması gereken yere yerleştirilir.
  - Akış diyagramı:
    - N ve `Dizi[N]` okunur.
    - Dış döngü `I ← 2 || 1 || N`. Her turda `J = I`, `TMP = Dizi[I]`.
    - İç döngü while: `J > 1 AND TMP < Dizi[J−1]` olduğu sürece `Dizi[J] = Dizi[J−1]`, `J = J − 1`. Yani TMP'den büyük elemanlar birer sağa kaydırılır.
    - While'dan çıkınca `Dizi[J] = TMP`.
  - İç döngü while'dır, çünkü her turda kaç kaydırma yapılacağı belli değildir.
  - Örnek `5 3 2 1 4 7 8`:
    - I = 2: 3, 5'ten küçük. 5 sağa kayar, 3 başa yazılır: `3 5 2 1 4 7 8`.
    - I = 3: 2, önce 5'ten, sonra 3'ten küçük. İkisi de kayar: `2 3 5 1 4 7 8`.
    - I = 4: 1 için 5, 3, 2 kayar: `1 2 3 5 4 7 8`.
    - Sonra 4, 5'in önüne geçer. 7 ve 8 yerinde kalır. Sonuç: `1 2 3 4 5 7 8`.
  - Avantajı: sıralı veya sıralıya yakın dizilerde yerindeki elemanlara dokunmaz. Sıralı dizide (`1 3 5 7 9`) while'a hiç girilmez. Her tur için yalnızca bir karşılaştırma yapılır, toplam N−1 karşılaştırma olur.
  - Verinin sıralıya yakın olduğu, çok karışık dağılmadığı bilinen durumlarda insertion sort kullanmak avantajlıdır.
- Büyük diziler için sıralama
  - Sıralı olup olmadığı bilinmeyen çok büyük dizilerde quick sort veya merge sort kullanılır.
  - Merge sort'un karmaşıklığı $N \log N$'dir. Dezavantajı, eleman sayısı kadar ek bellek gerektirmesidir (space complexity).
  - Heap sort da $N \log N$'dir. Hoca merge sort, quick sort ve heap sort için "böl ve yönet" (divide and conquer) der; heap sort büyük olasılıkla bu sınıfa girmez, yığın (heap) veri yapısına dayanır. Hepsinin birbirine göre küçük artıları ve eksileri vardır.
- Örnek: 8 bit gri seviyeli resmin histogramı (ekrandan, 01:26:25)
  - Soru: "8-bit gri seviyesindeki bir resmin bilgileri verilmektedir. Resmin histogramını çıkaran algoritmanın akış diyagramını çiziniz."
  - Histogram, her değerin kaç kez geçtiğini gösterir. Bir metinde histogram, her harften kaç tane olduğudur. Resimde ise her renk (gri tonu) değerinden kaç piksel olduğudur.
  - Gri seviye: 1 bit gri seviyede 0 siyah, 1 beyazdır. 2 bitte değerler 0 (siyah) ile 3 (beyaz) arasındadır. 8 bitte 0 (siyah) ile 255 (beyaz) arasındadır. Bit sayısı arttıkça renk derinliği artar.
  - Çözüm: sayaç dizisi (counter array).
    - N, M ve `resim[N][M]` okunur.
    - `I ← 0 || 1 || 255` ile `counter[I] = 0`. Sayaç dizisinin bütün gözleri mutlaka sıfırlanmalıdır.
    - `I ← 0 || 1 || N−1`, `J ← 0 || 1 || M−1`: `tmp = resim[I][J]`, `counter[tmp] = counter[tmp] + 1`.
    - `I ← 0 || 1 || 255` ile I ve `counter[I]` yazdırılır. Hangi renkten kaç tane olduğu birlikte yazılmalıdır.
  - Bu çözümde matris yalnızca bir kez gezilir.

- Sezar tipi şifreleme (2. ödevin üzerinden geçilmesi, ekrandan, 01:44:15)
  - Şifreleme (encryption) ve deşifreleme (decryption): her harf X kadar sağa (R) veya sola (L) kaydırılır. Örnek: `abcd`, 2 sağa kaydırılınca `cdef` olur.
  - Sorun alfabenin sonunda çıkar: `z + 1` tekrar `a` olmalıdır. Alfabe döngüsel bir yapıdır.
  - Yanlış formül: `(metin[i] + X) mod 26`. Karakterin ASCII değerinin modu alınır, harf alfabe içinde kalmaz.
  - Doğru formül: `(metin[i] - 'A' + X) mod 26 + 'A'`.
    - Önce harfin `'A'`'ya uzaklığı (0-25) bulunur, X eklenir, mod 26 alınır, sonra tekrar `'A'` eklenir.
    - Kontrol: `'Z' - 'A' + 2 = 25 + 2 = 27`, $27 \bmod 26 = 1$, `'A' + 1 = 'B'`.
  - Sola kaydırmada sonuç negatif olabileceği için mod almadan önce 26 eklenir: `(metin[i] - 'A' + X + 26) mod 26 + 'A'`.
  - X, 26'dan büyükse (ör. 50) birden fazla tur atılır. X'in de önce mod 26'sı alınabilir.
  - `metin[i] + (metin[i] - 'A' + X) mod 26` yanlıştır. `'A' + 2` için doğru sonucu verir, ama `'Z'` için `'Z' + 1` alfabenin dışına çıkar.
- Merge sort'un birleştirme (merging) adımı (ekrandan, 02:16:50)
  - Soru: iki sıralı diziyi tek bir sıralı dizide birleştiren algoritma. Örnek: `1 3 5 8 9` (N eleman) ve `2 4 6 7 11` (M eleman) → `1 2 3 4 5 6 7 8 9 11`.
  - Kötü çözüm: dizileri arka arkaya ekleyip sıralamak. Sıralama en iyi $N \log N$ (büyük olasılıkla sıralılığı bilinmeyen genel dizi için; insertion sort sıralı dizide $N - 1$ adım yapar), ortalama bir sıralama algoritmasında N^2 maliyetlidir.
  - İyi çözüm:
    - Birinci dizide I, ikinci dizide J indisi tutulur.
    - Her adımda `Dizi1[I]` ile `Dizi2[J]` karşılaştırılır. Küçük olan yeni diziye yazılır ve onun indisi bir artırılır.
    - `I < N` ve `J < M` olduğu sürece devam edilir.
    - Dizilerden biri bitince diğer dizinin kalan elemanları olduğu gibi yeni dizinin sonuna eklenir.
  - Karmaşıklık: N + M adım. Yazma sayısı tam N + M'dir, karşılaştırma sayısı büyük olasılıkla en fazla N + M − 1'dir.
  - Özel durum: bir dizinin bütün elemanları diğerininkilerden küçükse (`1 3 5 7` ve `8 9 10 11 12`), birinci dizi biter ve ikinci dizi olduğu gibi sona eklenir.

Hocanın Özellikle Vurguladığı Kısımlar

- Sıralama algoritmalarını iyi öğrenin ve kendi başınıza tekrar tekrar çözebilecek hâle gelin. İleride hep karşınıza çıkacak.
- Insertion sort'ta `Dizi[I]` mutlaka TMP'ye alınmalıdır. Kaydırma sırasında `Dizi[I]`'nın bulunduğu gözün üzerine yazılır ve değer kaybolur. While koşulu da `TMP < Dizi[J-1]` olmalıdır, `Dizi[I] < Dizi[J-1]` değil.
- Insertion sort'ta yer değiştirme (swap) yerine kaydırma yapılır. Yeni eleman kendisinden önceki bütün elemanlardan küçükse, o elemanların hepsinin bir sağa kayması gerekir. Bu, while döngüsünde kaydırmadan tek bir swap ile yapılamaz.
- Değişken adlarında tutarlı olun. Dizi `A` diye okunup algoritmanın geri kalanında `Dizi` diye kullanılırsa okuyan karışır.
- Birden fazla sayaca ihtiyaç varsa (2'den fazla bile olsa) sayaçları bir dizi olarak tutmak mantıklıdır. Sayaç dizisinin boyutu, sayılacak farklı değerlerin sayısı kadardır. İngilizce bir metindeki harfler için 26'lık bir dizi gerekir, ilk göz `A`'ya ve son göz `Z`'ye karşılık gelir.
- 256 ayrı değişkeni bir döngüyle tanımlayamazsınız. Bunun yolu bir dizi tanımlamaktır.
- Histogram için matrisi 256 kez, her renk için bir kez gezmek doğru sonuç verir ama gereksizdir. Matris bir kez gezilerek bütün renklerin sayısı bulunmalıdır.
- Akış diyagramında döngülerin 1'den başlaması zorunlu değildir. Doğru çizildiği sürece 0'dan başlayıp N−1'de bitebilir.
- `N, M, resim[N][M]` ifadesi akış diyagramında yalnızca okuma yapıldığını gösterir, bir tanım değildir.
- İkinci sınıfa kadar dizi ve matrisler hep sabit sayılarla tanımlanmalıdır, ör. `resim[100][100]`. Bazı derleyiciler `int resim[N][M]` yazımında hata vermez, ama ANSI C standardına göre statik tanımda boyut sabit olmalıdır. Daha sonra dinamik bellek ayırma (eleman geldikçe dizinin büyütülmesi) görülecek.

- Döngüsel yapılarda (baştan sona dönmeyi gerektiren yapılarda) kontrol koymak yerine modüler aritmetik kullanmak işi daha anlaşılır ve kolay kurgulanır hâle getirir.
- Mod ile if karşılaştırmasında kesin bir kazanan yoktur. Mod, bölme ve çarpma içerir. If bir kontrol ve dallanmadır. Hangisinin daha hızlı olduğu işlemci mimarisine (Intel, ARM, AMD) bağlıdır. Amaç, işlemleri olabildiğince basitleştirmek ve kontrol sayısını azaltmaktır. Bu konu alt seviye programlama dersinde 8086 mimarisiyle daha iyi oturacak.
- Yaptığınız algoritmaların kodlarını kendiniz yazın. Sorun yaşarsanız uygulama dersine getirin.

Kısa Tekrar Notları

- Insertion sort: `I` 2..N, `TMP = Dizi[I]`, `J = I`. `J > 1 AND TMP < Dizi[J-1]` sürece `Dizi[J] = Dizi[J-1]`, `J--`. Sonra `Dizi[J] = TMP`.
- Insertion sort sıralı dizide N−1 karşılaştırma yapar. Sıralıya yakın diziler için uygundur.
- Merge sort, quick sort, heap sort: $N \log N$. Merge ve quick sort böl ve yönet (heap sort büyük olasılıkla değil). Merge sort ek bellek ister.
- 8 bit gri: 0 siyah, 255 beyaz. 1 bit: 0 ve 1.
- Histogram: `counter[256]` sıfırla, matrisi bir kez gez, `counter[resim[I][J]]++`.
- Az sayıda farklı renk varsa: 2×K'lık bir tabloda (renk, sayı) yalnızca görülen renkler tutulur.
- Sezar şifreleme: `(c - 'A' + X + 26) mod 26 + 'A'`.
- İki sıralı diziyi birleştirme: iki indis, küçük olanı yaz, biri bitince diğerinin kalanını ekle. N + M yazma, büyük olasılıkla en fazla N + M − 1 karşılaştırma.

Detaylı Açıklamalar

Ders, uygulama dersinde başlanan insertion sort'un akış diyagramıyla başlar. Ardından insertion sort'un hangi durumlarda avantajlı olduğu ve büyük diziler için hangi algoritmaların tercih edildiği konuşulur. Aradan önce histogram problemi verilir, sonra sayaç dizisiyle çözülür. Sonra 2. ödevdeki şifreleme probleminin çözüm yolları tartışılır. Son bölümde merge sort'un birleştirme adımı fikir olarak konuşulur. Akış diyagramı bir sonraki derse kadar öğrencilere bırakılır.

Insertion sort: swap yerine kaydırma

Bir öğrencinin ilk çözümünde iç döngü `J > 1 AND Dizi[J] < Dizi[J-1]` olduğu sürece `Dizi[J]` ile `Dizi[J-1]`'in yerini değiştiriyordu. Bu çözüm çalışır, ama her adımda iki eleman yer değiştirir. Örneğin 2 yerleştirilirken önce 2 ile 5, sonra 2 ile 3 yer değiştirir.

Daha verimli yol, yerleştirilecek elemanı bir kez TMP'ye alıp ondan büyük elemanları birer sağa kaydırmak, sonunda TMP'yi boşalan yere yazmaktır. Böylece her adımda üç atama yerine bir atama yapılır (bu sayım ders sonrası eklemedir).

Diyagramın ilk hâlinde while koşulu `Dizi[I] < Dizi[J-1]` olarak yazılmıştı. Bir öğrenci bunun `TMP` olması gerektiğini fark etti. İlk kaydırmada `Dizi[J-1]`, `Dizi[I]`'nın bulunduğu göze yazılır ve `Dizi[I]`'nın değeri kaybolur. Bundan sonra karşılaştırma yanlış değerle yapılır. Hoca, analizi TMP üzerinden yaptığı için hatayı tahtada fark etmediğini söyledi.

Bir başka öğrenci kaydırma yerine yalnızca yan yana iki elemanı değiştirmeyi önerdi. Bu yeterli değildir. Örneğin `2 3 5` sıralı kısmından sonra 1 gelirse, 1'in en başa kadar gitmesi için 5, 3 ve 2'nin hepsinin bir sağa kayması gerekir.

Insertion sort'ta en iyi durum

Sıralı bir dizide (`1 3 5 7 9`) her turda `TMP < Dizi[J-1]` sorusu bir kez sorulur ve cevap hayırdır. While'a hiç girilmez. Bir öğrenci karşılaştırma sayısını önce 0 olarak söyledi. Hoca, while'a girilmese de koşulun kontrol edildiğini, yani her turda bir karşılaştırma yapıldığını vurguladı. Toplam N−1 karşılaştırma yapılır.

Histogram: neden sayaç dizisi?

Bir öğrenci 256 renk için 256 ayrı değişken tanımlamayı önerdi. Böyle değişkenler bir döngüyle tanımlanamaz. Her birini elle tanımlamak da çok zahmetlidir. Başka bir öğrencinin önerisiyle 256 gözlü bir sayaç dizisi kullanıldı. Dizinin K. gözü, K değerli renk için bir sayaçtır. Eskiden bir kelimedeki harfleri sayarken tek bir sayaç kullanılıyordu. Burada aynı mantık birden fazla sayaç için bir diziyle uygulanır.

Örneğin resmin ilk satırında `0 5 0 10 0` varsa: 0 görülünce `counter[0]` 1 olur, 5 görülünce `counter[5]` 1 olur, ikinci 0 ile `counter[0]` 2 olur. Böyle devam eder.

Bir öğrenci, sayaç dizisini sıfırlayan döngünün içine matris döngülerini koymuştu: her renk için matrisi baştan gezip o rengi sayıyordu. Sonuç doğrudur, ama matris 256 kez gezilir. İstenen, matrisi bir kez gezip bütün renkleri birlikte saymaktır.

Bir öğrenci, sayısı 0 olan renklerin yazdırılmayabileceğini söyledi. Bu, bir başka sorunu açar: resimde yalnızca 4-5 farklı renk varsa 256'lık alanın çoğu boşa ayrılır. 24 bitlik renkli bir resimde 2^{24}'lük bir dizi gerekir ve bu çok anlamsız olur. Bu durumda 2×K'lık bir tablo kullanılabilir:

1. Resim taranır. İlk değer 0'dır. Tabloda 0 yoktur. 0 eklenir ve sayısı 1 yazılır.
2. Sonra 5 gelir. 5 tabloda yoktur. 5 eklenir ve sayısı 1 yazılır.
3. Yeniden 0 gelir. 0 tabloda vardır, sayısı 2'ye çıkar.
4. Sonra 10 eklenir, bir 0 daha gelince 0'ın sayısı 3 olur.

Böylece yalnızca resimde geçen renkler için yer kullanılır. Bunun bedeli, her yeni değerde tablonun aranmasıdır.

Şifreleme: mod nerede alınmalı?

Sınıfta şu formüller tartışıldı:

Formül	Sonuç
`(metin[i] + X) mod 26`	Yanlış. ASCII değerinin modu alınır, harf alfabede kalmaz.
`(metin[i] − 'A' + X + 26) mod 26 + 'A'`	Doğru. +26 sola kaydırmada negatif değerleri önler.
`metin[i] + (metin[i] − 'A' + X) mod 26`	Yanlış. `'Z'` için `'Z' + 1` olur.
`(metin[i] − 'A' + X) mod 26 + 'A'`	Sağa kaydırma için doğru.

Bir öğrenci önce farkı hesaplayıp sonucun 0-25 aralığında olup olmadığını if ile kontrol ettiğini söyledi. Hoca, kontrol yerine doğrudan mod almanın yeterli olduğunu gösterdi. Büyük ve küçük harflerin ikisi de varsa, her grup kendi başlangıç harfine (`'A'` veya `'a'`) göre işlenmelidir.

Ders sonrası ekleme (derste yapılmadı): kontrolde `ABCDZYX` 2 sağa kaydırılınca `CDEFBAZ`, geri 2 sola kaydırılınca tekrar `ABCDZYX` oldu. 1 sola kaydırılınca `ZABCYXW`, `'A'` 53 kaydırılınca `'B'` elde edildi. C'de `%` operatörü negatif sayılarda negatif sonuç verebildiği için +26 eklemek gerçekten gereklidir.

Birleştirme (merging): sınır kontrolü

`1 3 5 8 9` ve `2 4 6 7 11` için: 1 < 2 olduğu için 1 yazılır ve I ilerler. 3 > 2 olduğu için 2 yazılır ve J ilerler. 3 < 4, 5 > 4, 5 < 6, 8 > 6, 8 > 7, 8 < 11, 9 < 11 karşılaştırmalarıyla devam edilir. Birinci dizi bitince ikinci dizide kalan 11 sona eklenir.

I'nın dizinin sonuna gelip gelmediği N ile, J'ninki M ile kontrol edilir. Dizilerin uzunluklarının farklı olması sorun çıkarmaz. Hangi dizi önce biterse diğerinin kalanı olduğu gibi eklenir.

Bir öğrenci alternatif olarak iki dizinin sonuna çok büyük bir değer (ör. 1000) eklemeyi önerdi. Böylece diziler hiçbir zaman "bitmez" ve ayrı bir sınır kontrolü gerekmez. Hoca bunun da uygulanan yöntemlerden biri olduğunu söyledi ve öğrencilerin üzerine düşünmesini istedi. Birleştirme algoritmasının akış diyagramı, bir sonraki derse kadar öğrencilere bırakıldı.

Kontrol

Ders sonrasında aşağıdaki kodlar `gcc -Wall` ile derlendi ve denendi. Insertion sort `5 3 2 1 4 7 8` dizisini `1 2 3 4 5 7 8` olarak sıraladı. Birleştirme iki örnekte de doğru sonucu verdi.

```
void insertion(int dizi[], int N)
{
    int i,j,tmp;
    for(i=1;i<N;i++)
    {
        tmp=dizi[i];
        j=i;
        while(j>0 && tmp<dizi[j-1])
        {
            dizi[j]=dizi[j-1];
            j=j-1;
        }
        dizi[j]=tmp;
    }
}

char kaydir(char c, int X)          /* X > 0 sağa, X < 0 sola */
{
    X=X%26;
    return (c-'A'+X+26)%26+'A';
}

int birlestir(int a[], int N, int b[], int M, int c[])
{
    int i=0,j=0,k=0;
    while(i<N && j<M)
    {
        if(a[i]<b[j]) c[k++]=a[i++];
        else c[k++]=b[j++];
    }
    while(i<N) c[k++]=a[i++];
    while(j<M) c[k++]=b[j++];
    return k;
}
```

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.